实验报告

一、三种排序算法源码

```
#include <iostream>
#include <algorithm>
#include <functional>
#include <vector>
#include <iterator>
#include <ctime>
#include <cstdlib>

const int len=100000;

using namespace std;

int part(vector<int> & v,int left,int right)
{
	int i=left,j=right,pivot=v[i];//以首元素为基准

	while(i<j)
	{
		while(i<j && v[j]>pivot)//从右向左遍历，找到第一个小于等于 pivot 的数
			j--;
		if (i<j)//如果相等
			swap(v[i++],v[j]);//交换基准数与第一个比他小或等于的数的位置，并且 i 后移一位
		while(i<j && v[i]<=pivot)//从左向右遍历，找到第一个大于 pivot 的数
			i++;
		if (i<j)
			swap(v[i],v[j--]);//交换基准数与第一个比他大的数的位置，并且 j 向前移一位
	}
	return i;//返回最后的基准数所在的位置
}

void quickSort(vector<int> & v,int left,int right)
{
	int mid;
	if (left<right)
	{
		mid=part(v,left,right);//对原数组进行第一次划分，基准前的数都小于基准后的数
		quickSort(v,left,mid-1);//对基准左区间进行快速排序
```

```
            quickSort(v,mid+1,right);//对基准右区间进行快速排序
        }
}

int main()
{
        int iarray[100005];
        for (int i=0;i<len;i++)
            iarray[i]=rand()%len;//随机生成 100000 个数字


        //局部排序前 10 个，并进行输出
        vector<int> vector1(iarray,iarray+len);

        double startTime1=clock();//计时 1 开始

        partial_sort(vector1.begin(),vector1.begin()+10,vector1.end());

        double endTime1=clock();//计时 1 结束

        copy(vector1.begin(),vector1.begin()+10,ostream_iterator<int>(cout," "));
        cout<<endl;
        cout<<"The run time of Partial_Sort is:
"<<(double)(endTime1-startTime1)/CLOCKS_PER_SEC<<"s"<<endl<<endl;


        //整体排序，并输出前 10 个
        vector<int> vector2(iarray,iarray+len);

        double startTime2=clock();//计时 2 开始

        sort(vector2.begin(),vector2.end());

        double endTime2=clock();//计时 2 结束

        copy(vector2.begin(),vector2.begin()+10,ostream_iterator<int>(cout," "));
        cout<<endl;
        cout<<"The run time of Sort is:
"<<(double)(endTime2-startTime2)/CLOCKS_PER_SEC<<"s"<<endl<<endl;


        //自定义排序：快速排序，并输出执行时间
        vector<int> vector3(iarray,iarray+len);
```

```
    double startTime3=clock();//计时 3 开始

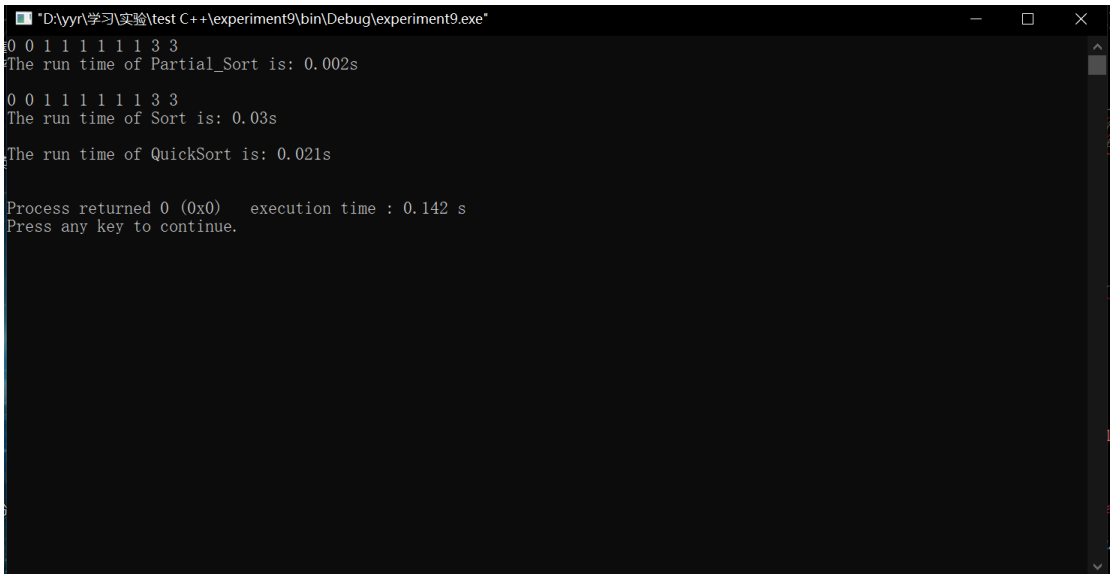
    quickSort(vector3,0,len-1);

    double endTime3=clock();//计时 3 结束

    cout<<"The run time of QuickSort is: "<<(double)(endTime3-startTime3)/CLOCKS_PER_SEC<<"s"<<endl<<endl;

    return 0;
}
```

二、输出图片



三、 总结

在 partial_sort()和 sort()中，由于 partial_sort()只进行局部排序，从 10 万个随机数字中得到最小的前 10 个，而 sort()是对所有的 10 万个数字进行排序之后再输出前 10 个。因此 partial_sort()的运行时间必定小于等于 sort()的运行时间。

自定义排序算法选择了快速排序，而 sort 并不是简单的快速排序，它对快速排序进行了优化。此外，它还结合了插入排序和推排序。系

统会根据数据形式和数据量自动选择合适的排序方法，它每次排序中不只选择一种方法。因此 STL 中的 sort()的运行时间必定小于等于简单的快速排序。